Selección sin ordenar

Adaptación a C#

No necesitas ordenar un arreglo para encontrar su mediana. No necesitas ordenar un arreglo para encontrar ningún rango — el elemento más pequeño, el más grande, el percentil 25, el 99. Dos algoritmos encuentran el k-ésimo elemento más pequeño de un arreglo en tiempo lineal, venciendo el coste $O(n \log n)$ del ordenamiento por un factor completo de $\log n$.

La lente local/global del capítulo anterior sigue aplicándose, con un giro. Quicksort, después de particionar alrededor de un pivote p, tiene que recurrir en *ambos* lados — ambas mitades necesitan que se arreglen sus inversiones locales. La selección solo tiene que recurrir en *un* lado: el lado que contiene el rango que buscas. El otro lado se queda sin ordenar para siempre y no te importa, porque la respuesta que quieres no está en él. Tirar una de las dos llamadas recursivas colapsa $O(n \log n)$ a $O(n)$.

El primer algoritmo — *quickselect* — hace esto con un pivote elegido al azar y te lleva a $O(n)$ en promedio, con una pequeña probabilidad de degenerar. El segundo — *mediana de medianas*, también llamado BFPRT por sus cinco inventores — elige su pivote con tanto cuidado que garantiza $O(n)$ incluso en el peor caso. El contraste entre ellos es el ejemplo más limpio que conozco de un compromiso recurrente en el diseño de algoritmos: la aleatorización te compra simplicidad; el determinismo te compra garantías, a costa de intrincación.

1. Quickselect: solo un lado importa

Empieza con el paso de partición del capítulo anterior. Después de que `Partition(items, lo, hi, order)` devuelve el índice final del pivote p:

- Todo elemento en `items[lo..p]` es menor que `items[p]`.
- Todo elemento en `items[p+1..hi]` no es menor.
- `items[p]` está en su posición final ordenada.

El tercer hecho es el que importa aquí. Si buscas el k-ésimo elemento más pequeño de `items[lo..hi]`, y la partición te dice que el pivote aterrizó en la posición p, tienes tres casos:

- $k = p$: el pivote *es* la respuesta. Devuelve `items[p]` y ya está.
- $k < p$: la respuesta está en algún lugar del lado izquierdo, `items[lo..p]`. Recurre allí.
- $k > p$: la respuesta está en algún lugar del lado derecho, `items[p+1..hi]`. Recurre allí.

En dos de los tres casos, recurres en solo un lado e ignoras el otro. Ese es el ahorro sobre quicksort, que siempre recurre en ambos.

Usaré un pivote elegido al azar — elegir un elemento uniformemente al azar del intervalo e intercambiarlo a la última posición antes de particionar. Es el truco de aleatorización estándar para quickselect, y es lo que hace que el análisis del caso promedio funcione limpiamente independientemente de la distribución de la entrada.

```
public static T QuickSelect<T>(
    IList<T> items, int k, Comparison<T> order = null, Random rng = null
        )
{
    order ??= Comparer<T>.Default.Compare;
    rng ??= Random.Shared;
    if (k < 0 || k >= items.Count)
        throw new ArgumentOutOfRangeException(nameof(k),
            $"k={k} fuera de rango para secuencia de longitud {items.
                Count}");
    return QuickSelectRange(items, 0, items.Count, k, order, rng);
}

private static T QuickSelectRange<T>(
    IList<T> items, int lo, int hi, int k,
    Comparison<T> order, Random rng)
{
    if (hi - lo == 1) return items[lo];

    // Pivote aleatorio: intercambia un elemento al azar a la ultima
        posicion.
    int pivotIdx = rng.Next(lo, hi);
    (items[pivotIdx], items[hi - 1]) = (items[hi - 1], items[pivotIdx]);

    int p = QuickSortImpl.Partition(items, lo, hi, order);
    if (k == p) return items[p];
    else if (k < p) return QuickSelectRange(items, lo, p, k, order, rng)
        ;
    else return QuickSelectRange(items, p + 1, hi, k, order, rng);
}
```

Listing 1: Quickselect

La envoltura pública hace la comprobación de límites y arranca la recursión. La recursión misma toma `lo` y `hi` explícitos para que pueda trabajar *in situ* sobre subrangos que se encogen sin copiar.

El cuerpo es reconociblemente quicksort, con dos llamadas recursivas colapsadas a una basada en en qué lado del pivote cae el rango objetivo k. El caso base es un solo elemento: si el intervalo tiene longitud 1, ese elemento *es* el k-ésimo más pequeño, por definición.

```
// Random.Shared no es sembrable, pero para reproducibilidad usamos
// una instancia con semilla fija.
var rng = new Random(42);

var items = new[] { 3, 1, 4, 1, 5, 9, 2, 6 };
Console.WriteLine(QuickSelectImpl.QuickSelect((int[])items.Clone(), 0,
    null, rng)); // 1
Console.WriteLine(QuickSelectImpl.QuickSelect((int[])items.Clone(), 3,
    null, rng)); // 3
Console.WriteLine(QuickSelectImpl.QuickSelect((int[])items.Clone(), 7,
    null, rng)); // 9
Console.WriteLine(QuickSelectImpl.QuickSelect((int[])items.Clone(),
    items.Length / 2, null, rng)); // 4
```

```
1
3
9
```

4

Ahora el análisis de coste. Con un pivote aleatorio, la posición esperada del pivote es el medio del intervalo. Así que en promedio cada llamada recursiva parte a la mitad el tamaño del intervalo: $T(n) = T(n/2) + O(n)$. El $O(n)$ es el coste de partición; la recursión hace el mismo trabajo sobre la mitad de los datos.

Desarrolla la recurrencia: $T(n) = O(n)+O(n/2)+O(n/4)+\cdots = O(n)\cdot(1+1/2+1/4+\cdots) = O(2n) = O(n)$. La serie geométrica es la magia. La primera partición cuesta n; la segunda cuesta $n/2$; la tercera $n/4$; y así sucesivamente. Suma la serie geométrica infinita y obtienes $2n$, que sigue siendo lineal. La recursión es logarítmica en profundidad pero el trabajo total es lineal, porque el trabajo en cada nivel se reduce a la mitad. La recurrencia de quicksort $T(n) = 2T(n/2) + O(n)$ no colapsa de esta forma porque las dos llamadas recursivas juntas hacen tanto trabajo como el padre, no la mitad. Matar una de esas llamadas es exactamente la diferencia entre $\log n$ niveles de trabajo n $(= n \log n)$ y $\log n$ niveles de trabajo que se encoge geométricamente $(= n)$.

Ese es el caso esperado. El peor caso, desafortunadamente, no es tan amable. Si sigues eligiendo el elemento más pequeño como pivote, la partición es desbalanceada — un lado tiene $n-1$ elementos, el otro tiene cero. La recursión se vuelve $T(n) = T(n-1) + O(n)$, que es $O(n^2)$. Con un pivote aleatorio, esta catástrofe requiere un adversario que prediga cada elección aleatoria, lo cual no puede, así que el comportamiento esperado domina con probabilidad abrumadora. Pero el peor caso no es $O(n)$; es $O(n^2)$.

Para obtener un peor caso $O(n)$ garantizado, necesitas elegir pivotes de una forma que no pueda ser frustrada por ninguna entrada. Esa es la mediana de medianas.

2. El pivote en el que puedes confiar: mediana de medianas

El problema con el peor caso de quickselect es el pivote. Un pivote "malo" — uno cerca del elemento más pequeño o más grande — deja una partición con casi todo de un lado, y la recursión hace $\Theta(n)$ trabajo sobre un intervalo apenas más pequeño que el original. Para obtener un $O(n)$ garantizado, necesitas un pivote que esté garantizado a estar en algún lugar del medio del arreglo — digamos, entre el percentil 30 y el 70 — de modo que cada llamada recursiva vea un intervalo de a lo sumo $0{,}7n$ elementos.

Una mediana sería ideal, pero encontrar una mediana es el problema que estás tratando de resolver. Eso suena circular, y casi lo es. El truco — el movimiento elegante del algoritmo BFPRT — es encontrar una *mediana aproximada* que sea suficientemente buena como pivote, usando una llamada recursiva sobre un problema más pequeño.

Aquí está la construcción.

1. Agrupa el arreglo en trozos de 5.
2. Encuentra la mediana de cada trozo (trabajo constante por trozo, ya que cada uno tiene solo 5 elementos).
3. Recoge las medianas de los trozos en una lista de tamaño $n/5$.
4. Encuentra recursivamente la mediana de esa lista. Llamémosla p.
5. Usa p como el pivote para particionar.

El pivote p es la *mediana de medianas* — el elemento medio de las medianas de los trozos. No es la mediana verdadera del arreglo original, pero está suficientemente cerca. La cota es:

Teorema 1. *Al menos el 30 % de los elementos son menores que p, y al menos el 30 % son mayores. Así que cualquier lado de la partición en el que tengas que recurrir, la llamada recursiva ve a lo sumo el 70 % de los elementos con los que empezaste.*

¿Por qué 30 %? Hay $n/5$ trozos. La mitad de ellos — es decir, $n/10$ trozos — tienen su mediana menor que p (ya que p es la mediana de las medianas de los trozos). En cada uno de esos trozos, la mediana del trozo es el tercero más pequeño de sus cinco elementos, lo que significa que dos elementos más en ese trozo también son menores que la mediana del trozo, y por tanto menores que p. Así que cada uno de esos trozos contribuye con 3 elementos menores que p: $n/10$ trozos $\times$ 3 elementos $= 3n/10$, que es el 30 %. El argumento simétrico da el 30 % mayor.

Esa es la garantía geométrica. Con el pivote p elegido así, la recursión satisface $T(n) = T(7n/10) + T(n/5) + O(n)$. El $T(n/5)$ es la llamada recursiva para encontrar la mediana de medianas; el $T(7n/10)$ es la selección recursiva sobre el mayor de los dos lados de la partición. Ambas fracciones son constantes, y su suma $7/10+1/5 = 9/10 < 1$, así que la recurrencia colapsa a $O(n)$ — el mismo truco geométrico que quickselect, solo que con la constante $9/10$ en lugar de $1/2$. La profundidad de la recursión es $O(\log n)$, pero el trabajo decae geométricamente a través de los niveles y el total es lineal.

El tamaño de trozo de 5 no es arbitrario. Más pequeño no funciona: con trozos de 3, el cálculo análogo da $T(n) = T(2n/3) + T(n/3) + O(n)$, cuyas fracciones suman 1, lo que falla en colapsar y da $O(n \log n)$. Los trozos de 5 son el tamaño impar más pequeño para el cual la suma de fracciones es estrictamente menor que 1. Trozos más grandes también funcionarían pero empeoran el factor constante — 5 da en el punto dulce.

Ahora el código. Escribiré la función de selección *fuera de lugar* — devuelve un valor en lugar de reorganizar la entrada — porque la claridad pedagógica vale el espacio extra $O(n)$. El movimiento que importa es la recurrencia, no el truco *in situ*.

```
public static T Select<T>(
    IReadOnlyList<T> items, int k, Comparison<T> order = null)
{
    order ??= Comparer<T>.Default.Compare;
    if (k < 0 || k >= items.Count)
        throw new ArgumentOutOfRangeException(nameof(k),
            $"k={k} fuera de rango para secuencia de longitud {items.
               Count}");
    var work = new List<T>(items);
    return SelectRange(work, k, order);
}

private static T SelectRange<T>(
    List<T> items, int k, Comparison<T> order)
{
    if (items.Count <= 5)
    {
        // Caso base: ordenar y elegir. InsertionSort es barato
        // en entradas diminutas.
        Basic.InsertionSort(items, order);
        return items[k];
    }

    // Encuentra un buen pivote via mediana de medianas.
    T pivot = MedianOfMedians(items, order);

    // Particion de tres vias por valor. La igualdad va a su propio
    // cubo para que el caso k-cae-en-la-racha-de-pivotes-iguales
    // tenga un hogar limpio.
    var lo = new List<T>();
    var eq = new List<T>();
    var hi = new List<T>();
    foreach (var x in items)
    {
```

```
        int c = order(x, pivot);
        if (c < 0) lo.Add(x);
        else if (c == 0) eq.Add(x);
        else hi.Add(x);
    }

    if (k < lo.Count)
        return SelectRange(lo, k, order);
    else if (k < lo.Count + eq.Count)
        return pivot;  // k cae dentro de la racha de pivotes iguales
    else
        return SelectRange(hi, k - lo.Count - eq.Count, order);
}
```

Listing 2: Selección determinista en $O(n)$ en el peor caso

La envoltura pública valida k y copia la entrada (para que la recursión pueda mutarla libremente sin sorprender a quien llama). La recursión hace el trabajo real. *Participación de tres vías por valor* (en lugar de la partición *in situ* de Lomuto del capítulo 4). Los tres cubos son `lo` (estrictamente menor que el pivote), `eq` (igual a él), y `hi` (estrictamente mayor). La recursión entonces sabe exactamente qué cubo contiene el rango k y cómo ajustar el valor de k para la llamada recursiva.

El pivote mismo viene de `MedianOfMedians`, que es el corazón del algoritmo:

```
private static T MedianOfMedians<T>(
    List<T> items, Comparison<T> order)
{
    var medians = new List<T>();
    for (int start = 0; start < items.Count; start += 5)
    {
        var group = items.GetRange(start, Math.Min(5, items.Count -
            start));
        Basic.InsertionSort(group, order);
        medians.Add(group[group.Count / 2]);
    }
    if (medians.Count == 1) return medians[0];
    // Recurre para encontrar la mediana de las medianas.
    return SelectRange(medians, medians.Count / 2, order);
}
```

Listing 3: `MedianOfMedians`: el corazón de BFPRT

Agrupa el arreglo en trozos de 5, ordena cada trozo con ordenamiento por inserción (que es genuinamente la herramienta correcta para una lista de 5 elementos — el análisis del capítulo 3 del ordenamiento por inserción en entradas pequeñas casi ordenadas aparece aquí como la herramienta correcta), recoge las medianas, y selecciona recursivamente la mediana de esa lista. La recursión toca fondo cuando hay cinco o menos medianas de trozos, momento en el cual el caso base de `SelectRange` ordena y devuelve.

```
var items = new[] { 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 8, 9, 7, 9, 3, 2,
    3, 8, 4 };
Console.WriteLine(SelectImpl.Select(items, 0));                    // 1
    (el mas pequeno)
Console.WriteLine(SelectImpl.Select(items, items.Length - 1));     // 9
    (el mas grande)
Console.WriteLine(SelectImpl.Select(items, items.Length / 2));     // 5
    (la mediana)

```

```
// Determinista: sin aleatoriedad, cada llamada devuelve la misma respuesta
Console.WriteLine(SelectImpl.Select(items, 7));                    // 3
Console.WriteLine(SelectImpl.Select(items, 7));                    // 3
```

```
1
9
5
3
3
```

3. Las tres preguntas, aplicadas

¿Es correcto?

Ambos algoritmos son correctos vía un argumento inductivo directo. Asume que la llamada recursiva devuelve la respuesta correcta sobre el subproblema más pequeño; entonces quickselect es correcto porque, después de particionar, la posición final del pivote p te dice exactamente qué lado del arreglo contiene el rango k, y la recursión desciende a ese lado con el rango sin cambios (o ajustado, en el caso del lado derecho, para tener en cuenta los elementos que se saltan). La mediana de medianas es correcta por el mismo argumento aplicado a su partición de tres vías: el rango k está o en `lo` (recurre, k sin cambios), o en el cubo `eq` (el pivote es la respuesta), o en `hi` (recurre con k desplazado por `lo.Count + eq.Count`). El caso base (un solo elemento o una lista de ≤ 5) es trivialmente correcto.

¿Cuán eficiente es?

Quickselect es $O(n)$ en tiempo esperado, con $O(n^2)$ en el peor caso en el evento desafortunado de pivotes consistentemente malos. El espacio es $O(\log n)$ en promedio — la pila de recursión — porque cada llamada recursiva trabaja sobre un subintervalo del mismo arreglo.

En palabras llanas para el caso esperado: cada nivel de recursión hace trabajo proporcional al tamaño actual del intervalo, y el tamaño esperado del intervalo se reduce a la mitad cada nivel. Suma $n + n/2 + n/4 + \cdots = 2n$. Lineal, con un factor constante de aproximadamente 2. El "2" es la suma de la serie geométrica — es por qué reducir a la mitad funciona y reducir a un tercio no.

La mediana de medianas es $O(n)$ en el peor caso. La recurrencia es $T(n) = T(n/5) + T(7n/10) + O(n)$, y como $1/5 + 7/10 = 9/10 < 1$, la suma geométrica converge y da $T(n) = O(n)$. El factor constante es mucho mayor que el de quickselect — aproximadamente $10\times$, porque el algoritmo hace la partición, más el cálculo de la mediana de medianas, más la selección recursiva. En la práctica, usas quickselect; la mediana de medianas existe para la pregunta del peor caso.

El espacio es $O(n)$ para la versión fuera de lugar implementada aquí (la partición de tres vías crea nuevas listas). Existe una versión *in situ* pero es lo suficientemente intrincada como para que habría oscurecido la recurrencia en el centro del algoritmo. La complejidad temporal asintótica es la misma en cualquier caso.

¿Es óptimo?

Sí, y el argumento es el más barato del capítulo: cualquier algoritmo que encuentre el k-ésimo elemento más pequeño de un arreglo debe inspeccionar cada elemento, porque si no lo hiciera, un adversario podría colocar la respuesta en la posición que se perdió. El mismo argumento que el argumento del adversario para búsqueda lineal del capítulo 1. Así que $\Omega(n)$ es la cota inferior; $O(n)$ es lo que tanto quickselect (en promedio) como la mediana de medianas (peor caso) logran. Estrechamente óptimos en ambos sentidos.

Teorema 2. *Encontrar el k-ésimo elemento más pequeño de un arreglo requiere $\Omega(n)$ inspecciones en el peor caso. Quickselect (en promedio) y la mediana de medianas (peor caso) logran $O(n)$.*

4. Recurrir en un solo lado

Quickselect y la mediana de medianas son correctos vía argumentos recursivos limpios — partición + recursión en el lado que contiene k, caso base trivial. Quickselect corre en $O(n)$ esperado con una pequeña probabilidad de $O(n^2)$; la mediana de medianas corre en $O(n)$ en el peor caso con un factor constante mayor. Ambos son óptimos — no puedes encontrar el k-ésimo elemento más pequeño de un arreglo sin inspeccionar cada elemento.

Cuando puedes recurrir en solo un lado en lugar de dos, el coste colapsa de $O(n \log n)$ a $O(n)$. La misma operación de partición — el mismo trabajo $O(n)$, la misma estructura de inversiones local-vs-global — te da $O(n \log n)$ si recurres en ambos lados (ordenamiento) y $O(n)$ si recurres en solo uno (selección). La diferencia entre $T(n) = 2T(n/2) + O(n)$ y $T(n) = T(n/2) + O(n)$ es exactamente la diferencia entre log-lineal y lineal. La suma geométrica que produce la segunda recurrencia es la expresión más limpia de "haz el trabajo, luego haz estrictamente menos de él, recursivamente" — y aparece una y otra vez en el resto del libro, cada vez que un algoritmo logra tirar la mayor parte de su trabajo restante después de una reducción por fracción constante.

Quickselect es corto, legible, rápido en la práctica; el precio es una diminuta probabilidad de volverse cuadrático. La mediana de medianas es intrincada, cuidadosa y garantizada; el precio es un factor constante de $10\times$ en casos del mundo real. La mayor parte del código de producción usa quickselect (o algún híbrido). La mayoría de los cursos de algoritmos enseñan la mediana de medianas, porque la existencia de un algoritmo de selección lineal en el peor caso es uno de los resultados más sorprendentes del campo. Ambos vale la pena conocerlos.

En el siguiente capítulo, la pregunta cambia de nuevo. Hasta ahora todo algoritmo en este libro ha sido basado en comparaciones — lo único que sabe sobre su entrada es cómo comparar dos elementos. La cota inferior del árbol de decisión del capítulo 4 dijo que el ordenamiento basado en comparaciones no puede ser más rápido que $\Theta(n \log n)$. El siguiente capítulo te muestra que puedes ordenar en tiempo lineal si estás dispuesto a abandonar la restricción de solo comparaciones — usando los valores mismos como posiciones en memoria. Es el ejemplo más limpio del libro de cómo la restricción de un modelo dicta su cota inferior.

5. Código completo

```
using System;
using System.Collections.Generic;

namespace Codex.Select
{
    public static class QuickSelectImpl
    {
        public static T QuickSelect<T>(
            IList<T> items, int k, Comparison<T> order = null, Random
                rng = null)
        {
            order ??= Comparer<T>.Default.Compare;
            rng ??= Random.Shared;
            if (k < 0 || k >= items.Count)
                throw new ArgumentOutOfRangeException(nameof(k),
                    $"k={k} fuera de rango para secuencia de longitud {
                        items.Count}");
```

```
            return QuickSelectRange(items, 0, items.Count, k, order, rng
                );
        }

        private static T QuickSelectRange<T>(
            IList<T> items, int lo, int hi, int k,
            Comparison<T> order, Random rng)
        {
            if (hi - lo == 1) return items[lo];

            int pivotIdx = rng.Next(lo, hi);
            (items[pivotIdx], items[hi - 1]) = (items[hi - 1], items[
                pivotIdx]);

            int p = QuickSortImpl.Partition(items, lo, hi, order);
            if (k == p) return items[p];
            else if (k < p) return QuickSelectRange(items, lo, p, k,
                order, rng);
            else return QuickSelectRange(items, p + 1, hi, k, order, rng
                );
        }
    }
}
```

Listing 4: `QuickSelect.cs` — quickselect

```
using System;
using System.Collections.Generic;

namespace Codex.Select
{
    public static class SelectImpl
    {
        public static T Select<T>(
            IReadOnlyList<T> items, int k, Comparison<T> order = null)
        {
            order ??= Comparer<T>.Default.Compare;
            if (k < 0 || k >= items.Count)
                throw new ArgumentOutOfRangeException(nameof(k),
                    $"k={k} fuera de rango para secuencia de longitud {
                        items.Count}");
            var work = new List<T>(items);
            return SelectRange(work, k, order);
        }

        private static T SelectRange<T>(
            List<T> items, int k, Comparison<T> order)
        {
            if (items.Count <= 5)
            {
                Basic.InsertionSort(items, order);
                return items[k];
            }

            T pivot = MedianOfMedians(items, order);

            var lo = new List<T>();
            var eq = new List<T>();
            var hi = new List<T>();
```

```
            foreach (var x in items)
            {
                int c = order(x, pivot);
                if (c < 0) lo.Add(x);
                else if (c == 0) eq.Add(x);
                else hi.Add(x);
            }

            if (k < lo.Count) return SelectRange(lo, k, order);
            else if (k < lo.Count + eq.Count) return pivot;
            else return SelectRange(hi, k - lo.Count - eq.Count, order);
        }

        private static T MedianOfMedians<T>(
            List<T> items, Comparison<T> order)
        {
            var medians = new List<T>();
            for (int start = 0; start < items.Count; start += 5)
            {
                var group = items.GetRange(start, Math.Min(5, items.
                    Count - start));
                Basic.InsertionSort(group, order);
                medians.Add(group[group.Count / 2]);
            }
            if (medians.Count == 1) return medians[0];
            return SelectRange(medians, medians.Count / 2, order);
        }
    }
}
```

Listing 5: `Select.cs` — mediana de medianas (BFPRT)

6. Notas y lecturas adicionales

Quickselect es la invención de Tony Hoare en 1961, publicada como pieza complementaria del quicksort. La mediana de medianas es el algoritmo BFPRT, llamado así por sus cinco inventores: Manuel Blum, Robert Floyd, Vaughan Pratt, Ron Rivest y Robert Tarjan, que publicaron el resultado de selección lineal en el peor caso en 1973 — un artículo que aún vale la pena leer por la elegancia de la prueba. CLRS (4.ª ed.) §9.2 cubre quickselect; §9.3 cubre la mediana de medianas. *Algorithms* (4.ª ed.) de Sedgewick §2.5 cubre quickselect en forma amigable para producción. El análisis del tiempo lineal esperado del quickselect aleatorizado vía la recurrencia del tamaño de partición está en CLRS §9.2; *Randomized Algorithms* (1995) de Motwani y Raghavan da un tratamiento probabilístico más completo. La elección del tamaño de grupo 5 en la mediana de medianas es célebamente el grupo impar más pequeño que hace que las fracciones recursivas sumen menos de 1; el tamaño de grupo 3 falla, el tamaño de grupo 7 funciona pero con una constante peor.